Source: docs\LOADING_EXPERIENCE.md | Export: 11 November 2026

Loading experience

Local presentation update, verified 9 November 2026. No production deployment, payment, Firebase data, rules or environment changes are included.

Shared components

`src/loading/Loading.tsx`, `Indicators.tsx` and `loading.css` provide:

- `MorphingInfinity`: the supplied five-second path morph, decorative within a named status region. Uses the already installed `motion/react` dependency.
- `PageLoader`: centred infinity indicator for access and lazy Admin loading.
- `Loading` / `SectionLoader`: one named status and a content-shaped skeleton; slow-read guidance after five seconds; existing offline Retry retained.
- `Skeleton`, `SkeletonLayout`, `ProductCardSkeleton`, `ProductGridSkeleton`, `ProductDetailsSkeleton`, `OrderListSkeleton`, `TableSkeleton`, `FormSkeleton`, `DashboardSkeleton` and `ChartSkeleton`: inert decorative placeholders.
- `InlineLoader` / `ButtonLoader`: compact feedback with the original action label.
- `PaymentVerificationLoader`: a slowly orbiting shield/lock, never a success animation. Server-authoritative payment state still decides confirmation.
- `ImageUploadLoader`: actual upload progress, indeterminate if no finite progress exists. Reaching 100% means transfer progress, not trusted media verification.
- `PreviewImage`: keeps the original img, dimensions, alt text and crop; its empty box has a subtle loading background, removed on load/error or after 15 seconds. Existing fallback handlers remain intact; photos are never tinted.
- Existing `ImageState`, `Recovery`, `Updating` and `ConnectionNotice` remain the shared image, error, retained-refresh and offline presentation.

Before the React bundle mounts, `index.html` contains a small infinity trace and connection/reload guidance. React replaces it when mounted. Reduced motion uses static artwork; no JavaScript-disabled storefront is promised.

Async presentation audit

The routed frontend and its providers, shared clients, read hooks, editors and dialogs were inspected. Existing suitable skeletons were retained rather than rewriting working reads. Coverage below includes retained and newly connected UI.

Area	Initial read / lazy state	Small actions and recovery
Home	Banner, collections, product grids, campaign sections	Retained refresh, image fallback and Retry
Shop, categories, search, campaigns	Product/header grids, detail gallery/info	Inline refresh/load-more, empty/error distinction
Bag and Wishlist	Item/summary and product-card skeletons	Quantity/removal/variant actions and retry
Delivery, review, payment	Address, item, pricing/summary placeholders	COD submission, online restoration/verification, coupons and points
Authentication and account	Session/access skeleton, profile/form skeletons	Registration/login/reset/sign-out/profile save indicators
Addresses, points, My Orders, order details	Address, ledger, order-card and detail skeletons	History pagination, requests, payment recheck and retry
About, Contact, policies	Editorial skeletons, image loading	Contact form remains usable while contact metadata loads; send indicator
Admin access / lazy route	Named centred infinity loader	Existing auth timeout, retry and denial handling
Admin dashboard	Skeletons inside actual metric cards, recent-order/stock tables	Existing refresh/errors retained
Admin Products and Categories	Table/card, editor/category-read placeholders	Save/publish/visibility and signed-image upload indicators
Admin orders and delivery	Table/detail/form skeletons	Bulk updates, individual actions, delivery save and payment reconciliation
Admin customers	Directory/detail/address/order-history placeholders	Export shows real prepared-record count; cancel/retry preserved

Area	Initial read / lazy state	Small actions and recovery
Inventory and audit history	Responsive table/form skeletons	Stock adjustment, conflict/error recovery
Coupons, campaigns and banners	Table and editor skeletons	Save/disable, image transfer and verification feedback
Website Content, About, Contact Information	Form/section skeletons	Save/conflict feedback, founder/media previews
Store Settings, login appearance, shipping, theme	Form/section skeletons	Save/theme controls, upload/preview feedback
Notifications	Loading list within the existing popover	Mark-read indicator and existing retry
Legacy reports component	Summary/table skeleton	Existing report error handling

The current dashboard/reports do not contain real chart widgets. `ChartSkeleton` is available for real chart consumers; this task does not add fictitious charts or numbers. Unrouted prototype/demo screens were not redesigned.

Behaviour and accessibility

- Existing Lora, storefront themes and Admin styling are preserved. Loading surfaces inherit storefront surface, border, primary and text variables; Admin gets the existing ivory/maroon defaults. No new dependency or theme.
- Skeletons are aria-hidden and contain no controls, fabricated prices or stock. Owning regions provide named statuses and aria-busy. Button indicators retain the original disabled conditions, labels, handlers and duplicate-submit locks.
- Reduced motion stops shimmer, spinner, orbit, preview and boot animation; the React infinity becomes a static path. A media-query subscription also handles preference changes while mounted, replacing the animated path so an already running morph stops immediately. Motion's installed hook only reads the initial preference, so it is not used for this indicator.
- Loaded content remains visible during background reads. Existing read deadlines, aborts, identity scoping, error/empty distinctions and manual retry are retained. No mutation replay, automatic order retry or fabricated percentage is added.
- All media signing/verification, claims plus enabled Admin access record, payment verification, reservations, coupon rules and loyalty rules are unchanged.
- Skeletons reserve approximate content geometry, with existing exact banner ratios and responsive product grids. Unknown record counts/content length can still change total page height when real data arrives; zero CLS is not claimed.

Verification

Executed locally on 9 November 2026:

- `npm run lint` (TypeScript; no separate ESLint configuration): passed.
- `npm test`: 189 passed, none failed or skipped.
- `npm run build`: passed; existing bundle-size warning remains.
- `npm run test:foundation`: SDK and empty-store development/preview browser checks passed.
- `npm run test:admin`: all 44 emulator tests and 31 existing browser files passed. The new loading-system browser file initially failed on its fixture; after correction it passed in a focused rerun. The full command was not repeated after those focused repairs.
- Loading suite: seven emulator checks, held/slow/offline read coverage, discovery, banner timing, customer history, Wishlist/Bag, customer order states and all checkout stages passed across the initial and focused reruns.
- Packaged Vercel catalog and payment runtime checks: passed. The generated artifact targets Node 24; these local invocations used Node 22.22.2.
- `git diff --check`: passed.

The new `scripts/loading-system-browser.test.py` exercises the real shared components in Chromium at 320, 390, 768 and 1440px in both themes, normal/reduced motion, upload progress, disabled actions, truthful payment state and held authorised Admin reads. It runs within the existing emulator suite, alongside the payment browser checks. `scripts/loading-browser.test.py` additionally covers held requests, slow reads, deadlines, errors, offline, retained data, query changes, image failures and private reads without mutation replay. Existing image-upload browser tests retain their trusted-signing, failure, retry and save checks.

Browser fixture repairs retain every assertion: the boot test aborts a blocked bundle instead of holding document readiness open; the component harness uses Vite's actual default React exports; customer-history screenshots remove their own temporary style element instead of an unrelated last style tag. The motion regression verifies both enabling and disabling reduced motion while mounted.

Screenshots are local synthetic/emulator artifacts under `.cache/loading-system` and `.cache/loading-design`. They contain no production customers or payments. Mocked gateway/browser and emulator evidence does not prove hosted payment or deployment behaviour. Production acceptance remains a separate release step.

Representative artifacts: `.cache/loading-design/home-skeleton-390.png`, `.cache/loading-design/home-skeleton-1440.png`, `.cache/loading-system/admin-settings-390.png` and `.cache/loading-system/admin-settings-1440.png`. Both theme/component galleries are also in `.cache/loading-system`.

No known blank loading state remains in the audited routed flows. This does not promise zero layout shift for unknown record counts or future screens. No real Cloudinary upload, Razorpay payment or hosted acceptance was performed for this presentation task. No production environment, data, rules or application release was changed.